BCC_S32K144_Monitoring

Example project for Battery Cell Controller SW Driver
for MC33771C and MC33772C (Lite version)

Date: *15 December 2020*

Revision: *2.2*

Overview

The purpose of this example project is to demonstrate how to use the Lite version of Battery Cell Controller (BCC) SW Driver to get measurements and fault status information from the BCC using S32K144. Raw values of status registers and interpretation of measured values is printed to the console.

Hardware Requirements

Following is required:

- S32K144EVB-Q100 (MCU board)
- USB Micro B cable
- Communication type related EVBs:
    - SPI communication: FRDM33771CSPEVB or FRDM33772CSPEVB (BCC board)
    - TPL communication:
        - FRDMDUAL33664EVB (Board with two isolated network transceivers)
        - RD33771CDSTEVB or KIT33772CTPLEVB (BCC board)
- BATT-14CEMULATOR (Battery pack emulator for MC33771C) or BATT-6EMULATOR (Battery pack emulator for MC33772C)

Setting up Hardware

1. Make sure that you **disconnect** SBC (System Basis Chip) and power your S32K144EVB-Q100 from OpenSDA USB instead. SBC initialization and setting is not in scope of this example.

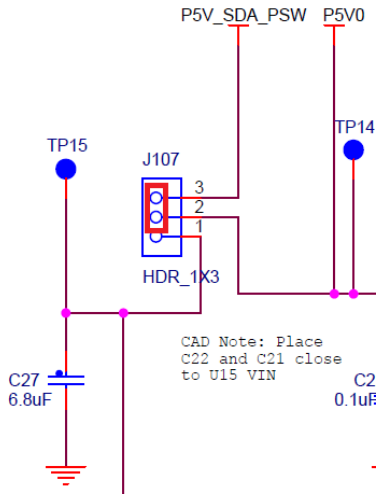

Figure 1. S32K144EVB-Q100 power source selection

2. **For SPI setup:** Attach the FRDM3377xCSPEVB board to the MCU board.

For TPL setup: Attach the FRDMDUAL33664EVB board to the MCU board and connect the BCC board (RD33771CDSTEVB or KIT33772CTPLEVB) to the first TPL interface of FRDMDUAL33664EVB board (J1 connector) by a twisted pair cable.

3. Setup the battery pack emulator according to its user manual and connect it to the BCC board using an appropriate cable.
4. Connect USB cable to USB interface on the MCU board and to your PC.

Figure 2 shows the complete HW setup for the TPL communication with MC33771C.

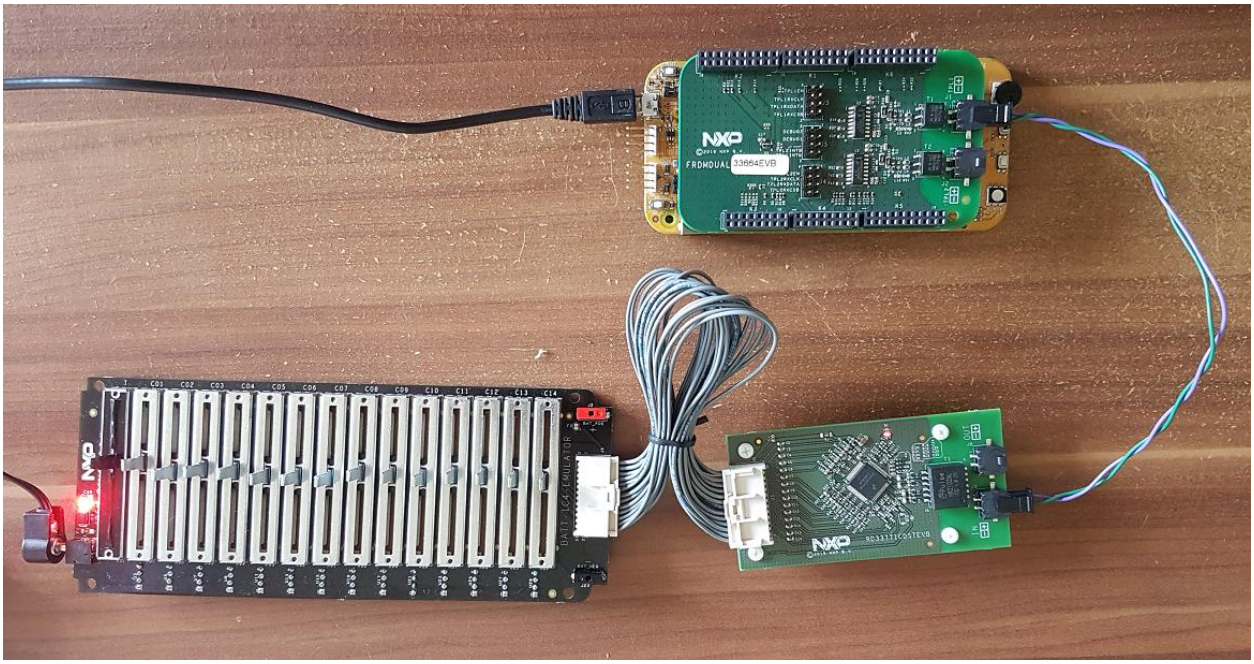

Figure 2. TPL HW setup

Setting up Software

Make sure that you have installed S32 Design Studio for ARM Version 2018.R1 with Update 11.

The application uses debug interface with virtual serial port to print user messages. Check that your debug connection has been set up properly. Type of used debug connection depends on used MCU. S32K144EVB-Q100 uses OpenSDA, see Figure 3. Note that number of COM port may differ because of different system resource usage. Baud rate is 115200 Bd.

Figure 3. OpenSDA virtual port

Description

The purpose of this example project is to demonstrate how to use the Lite version of the Battery Cell Controller SW Driver to get measurements and fault status information from the BCC device. Raw values of status registers and interpretation of measured values are printed to the console.

1. Initial configuration of BCC registers is printed.
2. Values of fuse mirror registers as well as BCC GUID is printed.
3. Following measured values (listed in ***bcc_measurements_t***) are printed.
    a. Coulomb counter registers (except MC33771C TPL board).
    b. ISENSE current (except MC33771C TPL board).
    c. Battery stack voltage.
    d. Battery cells voltage.
    e. Temperature on GPIOs configured as analog inputs.
    f. Internal circuit temperature.
    g. ADC1A and ADC1B voltage band gap reference.
4. Raw values of fault status registers (listed in ***bcc_fault_status_t***) is printed.

The project uses the following peripherals for **SPI setup**:

- LPSPI0 – SPI communication
- GPIO – RESET pin and red onboard LED
- LPUART1 – serial communication
- LPIT0 – timeout for the BCC SW driver

The project uses the following peripherals for **TPL setup**:

- LPSPI0 – SPI RX communication
- LPSPI1 – SPI TX communication
- GPIO – INTB pin, EN pin, TPL1+2TXCSB pin and red onboard LED
- LPUART1 – serial communication
- LPIT0 – timeout for the BCC SW driver

Pin selection for all mentioned peripherals is listed in Table 1 (SPI) and Table 2 (TPL).

Table 1. Pin selection for the SPI setup

Pin Function	S32K144
MISO	PTB3/LPSPI0_SIN
MOSI	PTB4/LPSPI0_SOUT
SCLK	PTB2/LPSPI0_SCK
CSB	PTB5
RESET	PTD4
Red on-board LED	PTD15
LPUART_RX	PTC6/LPUART1_RX

LPUART_TX	PTC7/LPUART1_TX

Table 2. Pin selection for the TPL setup

Pin Function	S32K144 pin
TPL1TXDATA	PTB16/LPSPI1_SOUT
TPL1TXSCLK	PTB14/LPSPI1_SCK
TPL1TXCSB	PTA6/LPSPI1_PCS1
TPL1+2TXCSB	PTD3
TPL1RXDATA	PTB3/LPSPI0_SIN
TPL1RXSCLK	PTB2/LPSPI0_SCK
TPL1RXCSB	PTB5/LPSPI0_PCS1
TPL1EN	PTB0
TPL1INTB	PTB1
Red on-board LED	PTD15
LPUART_RX	PTC6/LPUART1_RX
LPUART_TX	PTC7/LPUART1_TX

Example project consists of several files/folders apart from main.c as you can see in Figure 4.

- **Sources/bcc** contains the BCC SW driver for MC33771C and MC33772C.
- **Sources/bcc_s32k144** contains external functions of BCC SW driver related to this example and S32K144 MCU.
- **Sources/utils** contains files used for serial communication.
- **Sources/main.c** contains functions for setup of used MCU peripherals and functions implementing functionality described above.
- **Sources/common.*** files contain common functions and definitions as well as initial BCC device register values.
- **Sources/monitoring.*** files contain functions for doing a measurement and printing results to the console.

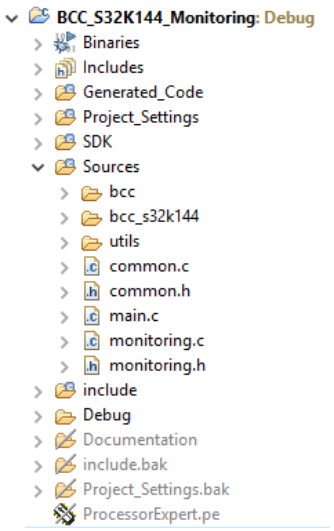

Figure 4. Example Code Structure

Additionally configuration of several used MCU peripherals is done by Processor Expert components. Outputs of those are generated configuration structures (in *Generated_Code* folder) which are then passed to initialization functions in *main.c*.

Import the Example Project

The following steps shows how to import an example project into S32 Design Studio for ARM Version 2018.R1.

1. In S32 DS click on the ***File / Import***.
2. Choose ***General / Existing Projects into Workspace***.
3. Click ***Browse to select root directory*** with your downloaded example projects.
4. ***Select project*** named **BCC_S32K144_Monitoring** and click ***Finish*** to complete the process.
5. At the top of **Sources/common.h** file define either macro **SPI** (in case of SPI HW setup) or **TPL** (in case of TPL HW setup).
6. At the top of **Sources/common.h** file define either macro **MC33771C** (in case of HW setup with MC33771C) or **MC33772C** (in case of HW setup with MC33772C).
7. Now the example project should be in your workspace and ready to run.

Note: In case of the TPL HW setup, it is also possible to change the number of BCC devices in the chain by macro **BCC_DEMO_BCC_CNT** at the top of **Sources/common.h**. (one BCC device is considered by default). It is also possible to mix MC33771C and MC33772C devices in the TPL chain when you change the BCC configuration (**bcc_drv_config_t** structure) in **initDemo()** in **main.c**.

Building and Running the Project

In order to build and run the project you need to ***build*** the project in a usual way. If the build is successful, ***debug and run*** the project. This can be accomplished in following steps:

1. Click on the **arrow** next to the **debug icon** and select **Debug Configurations**.
2. **Select** one of the existing configurations with **project name** under **PEMicro** group or **create** one by double clicking on this group.
3. Pick up proper **debug interface** and **USB port**.
4. Apply changes and click on **Debug**.

Note that the first build shows several warnings related to implicit declaration of math functions (Figure 5). These warnings have no impact on example functionality and will disappear after the next build.

Problems | Tasks | Console | Properties | Debugger Console | Search | Progress

0 errors, 8 warnings, 0 others

Description	Resource	Path	Location	Type
Warnings (8 items)				
implicit declaration of function 'pow' [-Wimplicit-function-declaration]	nxp_console.c	/BCC_S32K144_Monitoring/Sources/utils	line 620	C/C++ Probl
implicit declaration of function 'strtod' [-Wimplicit-function-declaration]	nxp_console.c	/BCC_S32K144_Monitoring/Sources/utils	line 1597	C/C++ Probl
incompatible implicit declaration of built-in function 'pow'	nxp_console.c	/BCC_S32K144_Monitoring/Sources/utils	line 620	C/C++ Probl
incompatible implicit declaration of built-in function 'pow'	nxp_console.c	/BCC_S32K144_Monitoring/Sources/utils	line 628	C/C++ Probl
implicit declaration of function 'exp' [-Wimplicit-function-declaration]	monitoring.c	/BCC_S32K144_Monitoring/Sources	line 400	C/C++ Probl
implicit declaration of function 'round' [-Wimplicit-function-declaration]	monitoring.c	/BCC_S32K144_Monitoring/Sources	line 401	C/C++ Probl
incompatible implicit declaration of built-in function 'exp'	monitoring.c	/BCC_S32K144_Monitoring/Sources	line 400	C/C++ Probl
incompatible implicit declaration of built-in function 'round'	monitoring.c	/BCC_S32K144_Monitoring/Sources	line 401	C/C++ Probl

Figure 5. Warnings after the first build

If you have any questions related to how to work with debug configurations, see ***S32 Design Studio User's Guide***.